A. Comparison with Existing Compilers

Existing lattice-surgery compilers include scalable instruction-level compilation [1] and dependency-aware mapping and routing [2]. Here we compare our compiler with TopoLS [3], which simplifies a ZX graph before embedding it in three-dimensional spacetime, whereas our compiler optimizes spacetime paths on a regular patch layout. Both outputs can be represented as TQEC BlockGraphs [4], enabling their occupied geometries to be evaluated in a common representation. The path search in our double-slice router is temporally local, spanning only two adjacent code beats, unlike whole-circuit transformations that require the circuit fragment to be available before embedding. Although the present scheduler uses whole-circuit critical-path information to prioritize ready instructions, the routing stage could in principle operate online using a bounded buffer of upcoming instructions. Such an extension would be useful for circuits containing classically conditioned Pauli corrections after T-gate reduction, as well as programs whose subsequent quantum instructions are selected or generated during execution.

We use TopoLS revision `19d1e1e`.

We use eight pairs of eight-qubit, 32-CNOT synthetic circuits generated with the max-parallel and min-parallel generators of MQT QECC [5]. These families follow the contrasting initial parallelism studied in the MQT QECC compilation benchmarks [6, Section VII.B], with the same scaling of four CNOTs per qubit.

For our compiler, we use SA-based mapping followed by double-slice routing with instruction look-ahead and kink-parity correction. TopoLS uses its Python engine and default search settings, including ZX and direction optimization and a five-seed search. Both compilers are allocated the same 7×7 spatial footprint while retaining their respective placement constraints and search strategies.

We report total execution time in code beats, following the metric used throughout this paper, and convert both outputs to TQEC BlockGraphs for common volume accounting. For these circuits, the volume counts computation and storage cubes, with ports and pipes contributing zero volume. Figure 2 shows representative spacetime geometries, and Table I reports aggregate results.

Across the eight paired seeds, double-slice routing reduces occupied volume by approximately 47% for max-parallel circuits and 39% for min-parallel circuits. The corresponding total-execution-time results in Table I show the same overall trend across both families. Thus, under the matched spatial allocations used here, bounded-depth routing consistently produces more compact spacetime geometries.

[1] G. Watkins, H. M. Nguyen, K. Watkins, S. Pearce, H.-K. Lau, and A. Paler, A High Performance Compiler for Very Large Scale Surface Code Computations, Quantum **8**, 1354 (2024).

[2] A. Molavi, A. Xu, S. Tannu, and A. Albarghouthi, Dependency-Aware Compilation for Surface Code Quantum Architectures, Proc. ACM Program. Lang. **9**, 57 (2025).

[3] J. Zhou, Y. Liu, E. Decker, J. Kalloor, M. Weiden, K. Chen, C. Iancu, and G. Li, TopoLS: Lattice Surgery Compilation via Topological Program Transformations (2026), arXiv:2601.23109 [quant-ph].

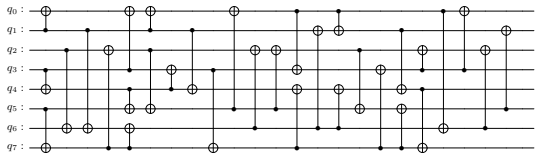
[4] A. Suau, Y. Zhang, P. Thakre, Y. Zhao, K. Dubey, J. A. Bolanos, A. Schelpe, T. Hao, P. Seitz, G. G. Guerreschi, Á. E. Á. Pérez, R. Stahn, J. Lenssen, B. Reid, and A. Fowler, Tqec: A Python package for topological quantum error correction, Journal of Open Source Software **11**, 9142 (2026).

[5] R. Wille, L. Berent, T. Forster, J. Kunasaikaran, K. Mato, T. Peham, N. Quetschlich, D. Rovara, A. Sander, L. Schmid, D. Schönberger, Y. Stade, and L. Burgholzer, The MQT Handbook : A Summary of Design Automation Tools and Software for Quantum Computing, in *2024 IEEE International Conference on Quantum Software (QSW)* (2024) pp. 1–8.

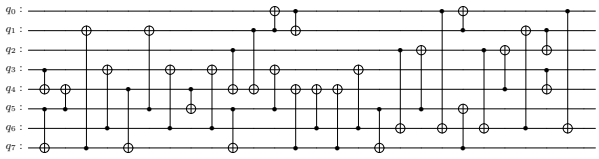
[6] L. S. Herzog, L. Berent, A. Kubica, and R. Wille, Lattice Surgery Compilation Beyond the Surface Code, in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 01 (2025) pp. 489–500.

TABLE I: Results over 8 paired circuit-generation seeds, reported as mean ± sample standard deviation. Total execution time is reported in code beats by each compiler. Occupied volume is measured from the exported TQEC BlockGraph.

Circuit	Method	n	Total execution time	Occupied volume
Max-parallel	Double-slice	8	11.4 ± 0.9	265.1 ± 15.1
Max-parallel	TopoLS	8	22.2 ± 3.7	500.1 ± 44.1
Min-parallel	Double-slice	8	14.0 ± 1.1	282.6 ± 15.1
Min-parallel	TopoLS	8	21.4 ± 3.4	467.2 ± 56.2

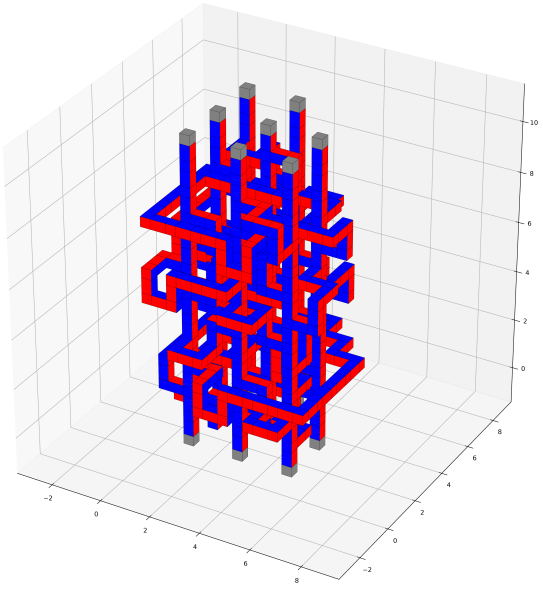


(a) Max-parallel.

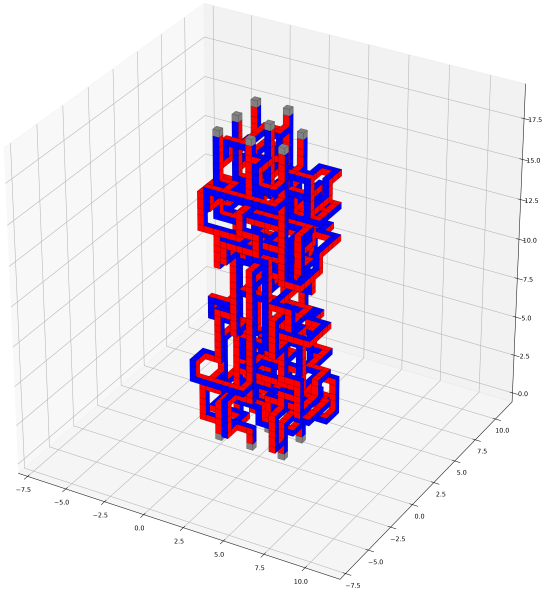


(b) Min-parallel.

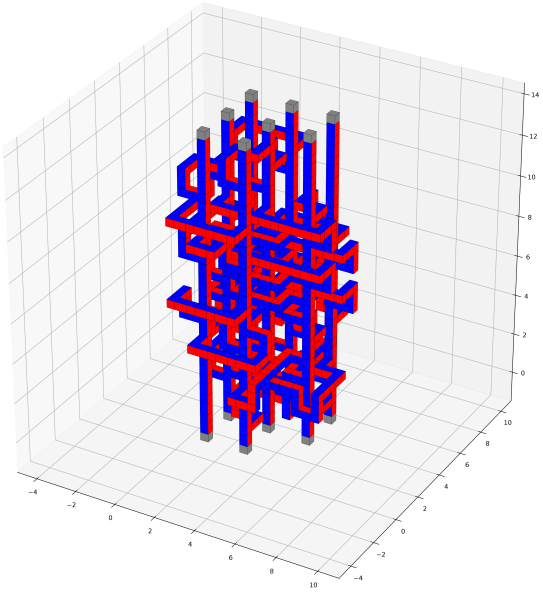
FIG. 1: Eight-qubit, 32-CNOT benchmark circuits generated with MQT QECC using input-generation seed 45.



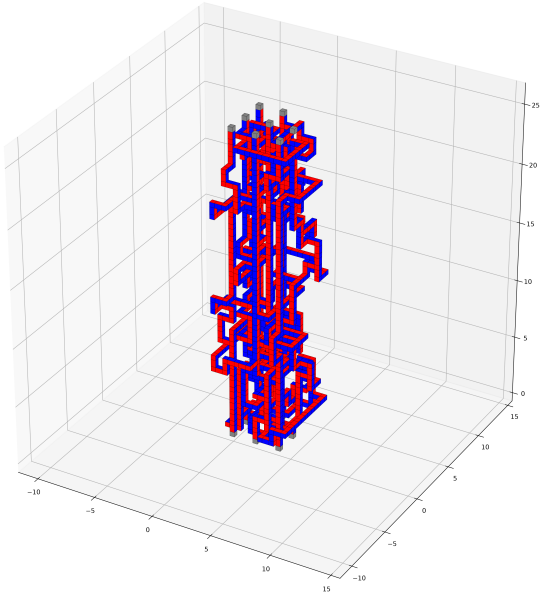
(a) Double-slice, max-parallel.



(b) TopoLS, max-parallel.



(c) Double-slice, min-parallel.



(d) TopoLS, min-parallel.

FIG. 2: Representative compiled spacetime geometries, rendered from TQEC BlockGraphs using TopoLS's renderer. The time direction is vertical, and blue and red faces indicate block boundary bases. Grey blocks indicate input/output ports and contribute zero volume. Each panel is independently fitted to its coordinate bounds, while Table I provides the aggregate quantitative comparison.